

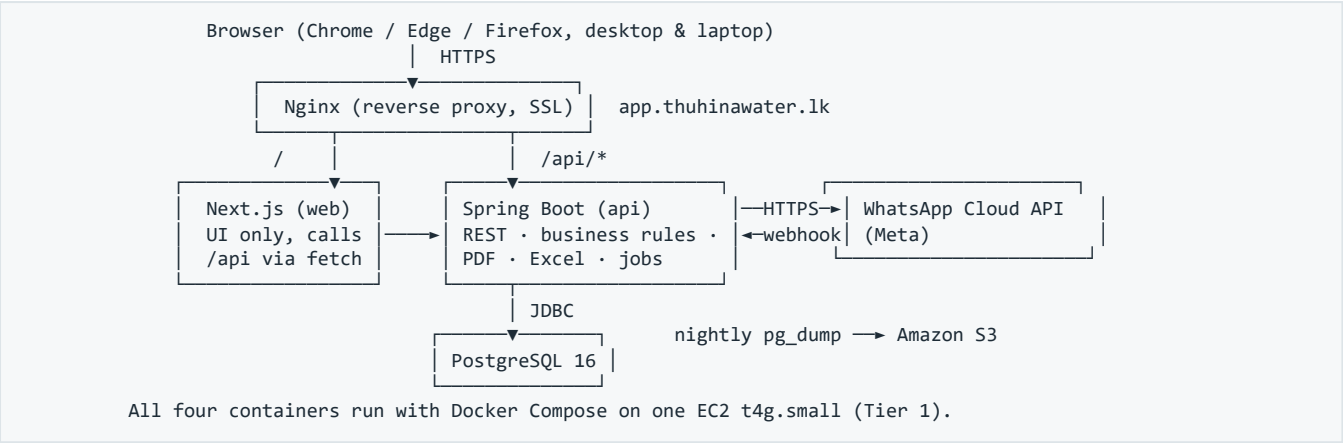
Thuhina Water – System Architecture & Design Document

Item	Detail
Product	Water Distribution Inventory & Sales Management System
Version	1.2
Date	09/10/2026
Prepared by	Amritha
Based on	BRD v4.0 (BRD Water Distribution System.md) · Clickable prototype (/prototype) · Technology Stack & Hosting (TechStack Thuhina Water.md)
Changes in 1.1	Client feedback 08/10/2026: agreed customer prices (Pricing Service, <code>customer_price</code>), customer status date, multi-bottle factory dispatch (<code>factory_dispatch</code>), running expenses (new module), customer quotations (new module); supplier quotation number renamed <code>SQ-</code>
Changes in 1.2	M00 build (09/10/2026): Spring Boot 4.1 instead of 3.x (3.x free support has ended); <code>login_attempt</code> table; settings rows keep who/when changed; numbering only moves forward for yearly / monthly sequences; API served under <code>/api</code> ; temporary password enforced by the server
Purpose	Defines how the system is built. It is the input for the Master Task Breakdown and for every module build.

Contents

1. Architecture overview
2. Technology choices
3. Code structure (repository, backend, frontend)
4. Data model
5. Core engines (business logic)
6. Module designs
7. API design
8. Security & roles
9. Background jobs & integrations (WhatsApp)
10. Printing & exports
11. Non-functional design
12. Environments, deployment & backup
13. Testing strategy & definition of done
14. Coding conventions
15. Traceability (modules ↔ BRD)
16. Risks & open points

1. Architecture overview



Key principles

#	Principle	Why
P1	All business rules live in the backend (Spring Boot). The frontend only shows data and calls the API.	One source of truth. A future delivery-staff mobile app reuses the same API (NFR-08).
P2	Every stock or bottle change goes through one Ledger Engine (5.1). No screen updates stock tables directly.	Keeps stock, customer bottle balances and money consistent (BR-12, BR-15).
P3	Saved transactions are never deleted or edited . Corrections are reversal entries (BR-14).	Audit and trust.
P4	Numbers and codes come from the Numbering Service (5.3) – never typed (FR-60, BR-17).	Unique, gap-free document numbers.
P5	Simple screens – one page per main activity, pop-ups / side panel for add-edit-view, only the next valid action shown (NFR-09).	Agreed with the client in the prototype.
P6	The prototype is the UI reference (layout, wording, validations) and <code>prototype/assets/js/store.js</code> is the reference implementation of the business rules .	It was tested end to end with the client.

2. Technology choices

Follows the NUVI shared stack (see TechStack Thuhina Water.md).

Layer	Choice	Notes
Frontend	Next.js 15 (App Router) + TypeScript	Client components for screens; no business logic
UI	Tailwind CSS + shadcn/ui	Dialog (pop-up), Sheet (side panel), Tabs, Table, Calendar
Data fetching	TanStack Query	Caching, refetch after save
Forms	React Hook Form + Zod	Same validation messages as the prototype
Dates	date-fns + react-day-picker	Display DD/MM/YYYY , calendar picker on every date field (NFR-07)
Backend	Spring Boot 4.1, Java 21	Spring Web MVC, Spring Security 7, Spring Data JPA (Hibernate 7), Validation, Scheduling; Jackson 3 for JSON. JWT with Nimbus (spring-security-oauth2-jose)

Layer	Choice	Notes
DB migrations	Flyway	Versioned SQL scripts, run on start-up
Database	PostgreSQL 16	NUMERIC for money, DATE for business dates
Auth	JWT in an httpOnly, Secure, SameSite=Strict cookie	Passwords hashed with BCrypt (NFR-05)
PDF	OpenPDF	Bills, invoices, statements, receipts, PO, delivery & load sheets
Excel	Apache POI	Report exports
API docs	springdoc-openapi (Swagger UI, disabled in production)	
Tests	JUnit 5, Testcontainers (PostgreSQL), MockMvc; Playwright for end-to-end smoke	
CI/CD	GitHub Actions → Docker images → UAT → Production	
Containers	Docker Compose: nginx, web, api, db	

Time zone: server and database run in `Asia/Colombo`. "Today" is the real current date. The fixed demo date (02/10/2026) was only for the prototype.

3. Code structure

3.1 Repository

```

Thuhina Water/
├─ server/           Spring Boot API (Java 21, Gradle)
├─ client/           Next.js web app (TypeScript)
├─ deploy/           docker-compose.yml, nginx.conf, backup scripts, .env.example
├─ doc/              BRD, tech stack, this document, task breakdown
├─ prototype/        approved clickable prototype (UI and rules reference)
└─ .github/workflows/ CI pipelines

```

3.2 Backend packages (`server/src/main/java/lk/thuhina/water/`)

Module-first structure (agreed). Each business module has its own package, and inside it the usual layers:

```

server/src/main/java/lk/thuhina/water/
├─ ThuhinaWaterApplication.java
├─ common/ config/ security/ numbering/ audit/ settings/ ← shared foundation
├─ customers/ ← example module
│   ├── controller/ CustomerController.java
│   ├── dto/         CustomerRequest, CustomerResponse, RegisterCustomerRequest ...
│   ├── service/     CustomerService.java, CustomerRegistrationService.java
│   ├── model/       Customer, CustomerUsualQty, CustomerBottleBalance, CustomerAccount
│   ├── repository/  CustomerRepository, CustomerBottleBalanceRepository ...
│   └── mapper/      CustomerMapper.java
├─ sales/            controller/ dto/ service/ model/ repository/ mapper/ + rules/ (pure business-rule classes)
├─ ledger/ masterdata/ partners/ inventory/ delivery/ purchasing/ production/
└─ billing/ expenses/ quotations/ reports/ printing/ whatsapp/ migration/ admin/

server/src/main/resources/
├─ application.yml, application-local.yml, application-prod.yml
├─ db/migration/      V001__foundation.sql, V002__admin.sql, V003__masterdata.sql ...
├─ db/seed/          sample data (local / UAT only)
└─ server/src/test/java/lk/thuhina/water/ ← same module folders (customers/, sales/ ...)

```

Layer rules: Controller → Service → Repository → Model. DTOs only at the controller boundary; entities are never returned by the API. Business rules live in services and `rules/` classes, never in controllers.

Services: one class per service (e.g. `CustomerService`) – **no interface + `ServiceImpl` pair by default**. An interface with implementations is used only where a second implementation is real: `WhatsAppClient` (Meta / test fake), `NotificationSender` (future email), `PdfRenderer`, `FileStorage` (local disk / S3 later).

Package	Contents
common	Base entity, money/date utils, error handling, paging, <code>BusinessException</code> (code + message)
config	Security, Jackson (DD/MM/YYYY is a UI concern; API uses ISO dates), OpenAPI, scheduling
security	JWT filter, login/logout, current user, permission constants
numbering	Numbering Service (5.3)
audit	Audit log writer + query
settings	Company details, business settings, feature flags (e.g. data migration visible, WhatsApp toggles)
admin	Users, roles
masterdata	Bottle types, products, customer types, areas, standard prices & deposits, old-bottle brands, Pricing Service (5.5)
partners	Suppliers (+ supplied items), filling factories (+ charges)
customers	Customers, usual quantities, bottle balances (read), status (active / inactive with date and reason), agreed customer prices (<code>customer_price</code>)
ledger	Ledger Engine (5.1): effects, stock balance, stock movements, reversals
inventory	Stock views, damage, adjustments, minimum levels
sales	Sales transactions (first purchase, exchange, left at door, extra, product, collect, customer damage, owed settlement), bill search
delivery	Schedule Service (5.4), daily list, delivery planning, bottle requirement
purchasing	Quotation requests, quotations, POs, goods receipts, supplier payments
production	Factory dispatches (all bottle types in one entry) → batches, returns, factory payments
billing	Customer accounts, aging, payments, receipts, monthly invoices, statements
expenses	Expense types, running expenses by month, month summary (feeds the Profit and Expenses reports)
quotations	Customer quotations (draft → sent → accepted / not accepted), print, register customer / apply agreed prices
reports	Dashboard + 10 reports, export
printing	PDF templates and renderers
whatsapp	Outbox, sender job, templates, webhook
migration	CSV import with validation preview (hidden until go-live)

3.3 Frontend structure (client/)

```
client/src/
├─ app/
│  └─ login/page.tsx
│  └─ (app)/
│     │                                     ← protected layout: sidebar + top bar
│     └─ dashboard/page.tsx
│     └─ master-data/bottles-products/page.tsx
│     └─ master-data/customers/page.tsx
│     └─ master-data/suppliers/page.tsx
│     └─ purchasing/page.tsx
│     └─ production/page.tsx
│     └─ inventory/page.tsx
│     └─ sales/enter-bills/page.tsx
│     └─ sales/bills/page.tsx
│     └─ sales/quotations/page.tsx
│     └─ delivery/planning/page.tsx
│     └─ delivery/daily-list/page.tsx
│     └─ billing/page.tsx
│     └─ expenses/page.tsx
│     └─ reports/page.tsx
│     └─ admin/page.tsx
│     └─ migration/page.tsx                (hidden by feature flag)
├─ components/
│  └─ layout/    AppShell, Sidebar (built from /api/auth/me menu), TopBar
│  └─ ui/        shadcn components
│  └─ shared/    DataTable, FormDialog, SidePanel, DateField (calendar, DD/MM/YYYY),
│                MoneyText (Rs. 1,350.00), StatusBadge, CustomerPicker (search by code/phone/name),
│                ConfirmDialog, ReasonDialog, PrintButton, StepBar, KpiCard,
│                PriceText (price + "agreed" badge + standard), CustomerPriceDialog, CustomerStatusDialog
├─ features/<module>/  API hooks (TanStack Query), forms, module components
└─ lib/              api client (fetch with credentials), formatters, zod schemas, permissions
```

Screen ↔ **prototype file map** (the prototype page is the visual spec):

Route	Prototype file
/dashboard	dashboard/dashboard.html
/master-data/bottles-products	master-data/bottles-products.html
/master-data/customers	master-data/customers.html (+ assets/js/customer-panel.js)
/master-data/suppliers	master-data/suppliers.html
/purchasing	purchasing/purchasing.html
/production	production/production.html
/inventory	inventory/inventory.html
/sales/enter-bills	sales/enter-bills.html
/sales/bills	sales/sales-bills.html
/sales/quotations	sales/quotations.html
/delivery/planning	delivery/delivery-planning.html
/delivery/daily-list	delivery/delivery-schedule.html
/billing	billing/billing.html
/expenses	expenses/expenses.html
/reports	reports/reports.html
/admin	admin/administration.html
/migration	migration/data-migration.html

4. Data model

4.1 Conventions

- Primary keys: `BIGINT` identity (`id`). Business codes (`C0001`, `B000001` ...) are separate **unique** columns.
- Money: `NUMERIC(12,2)` (Java `BigDecimal`). Quantities: `INTEGER`. Business dates: `DATE`. Timestamps: `TIMESTAMP`.
- Every table has `created_at`, `created_by`; editable master tables also `updated_at`, `updated_by`, `version` (optimistic locking).
- Enumerations are stored as `VARCHAR` with a `CHECK` constraint.
- Transaction tables are **insert-only** (no `UPDATE` except the `reversed_by_id` link and status fields on documents).

4.2 Entity overview



4.3 Tables

Administration & system

Table	Key columns
app_user	username (unique), full_name, phone, role (ADMIN / ACCOUNTANT / DELIVERY_STAFF), password_hash, must_change_password, active, last_login_at, token_version
audit_log	ts, user_id, username, role, action (CREATE / UPDATE / APPROVE / REVERSE / IMPORT / LOGIN / LOGOUT), entity, ref, details, ip – insert-only
doc_sequence	name (PK), prefix, next_value, padding, reset_rule (NONE / YEARLY / MONTHLY), period_key
app_setting	key (PK), value (JSON object), updated_at, updated_by – one row per group: company, business, features (later whatsapp)
login_attempt	username (PK, also for unknown usernames), failed_count, last_failed_at, locked_until – 5 failures → 15-minute lock; cleared on success and by the hourly job

Master data

Table	Key columns
bottle_type	code (PK, e.g. B20, generated from size), name, litres, active
product	code (P01), name, selling_price, cost_price, stock_qty (≥ 0), active

Table	Key columns
customer_type	name (unique), description, active
area	name (unique), active – used as the delivery route
price_entry	kind (WATER / DEPOSIT), bottle_type_code, customer_type_id (null for DEPOSIT), price, effective_from, reason – unique (kind, bottle_type_code, customer_type_id, effective_from)
old_bottle_brand	name, bottle_type_code, note, active, added_on

Partners

Table	Key columns
supplier	code (S01), name, contact, phone, email, address, terms_days (0, 7, 14, 30, 45, 60), active
supplier_item	supplier_id, item_type (BOTTLE / PRODUCT), item_code
factory	code (F01), name, address, contact, phone, email, licence, terms_days, active
factory_charge	factory_id, bottle_type_code, charge, effective_from

Customers

Table	Key columns
customer	code (C0001), name, address, area_id, phone, customer_type_id, delivery_day (1 = Mon ... 7 = Sun), cycle_weeks (1–4), next_delivery_date, pay_type (CASH / CREDIT / MONTHLY_BILL), credit_limit, terms_days, registered_on, status (ACTIVE / INACTIVE), status_date, status_reason, notes, whatsapp_opt_in, whatsapp_number
customer_usual_qty	customer_id, bottle_type_code, qty
customer_bottle_balance	customer_id, bottle_type_code, held, to_collect, owed – all CHECK (≥ 0); changed only by the Ledger Engine
customer_account	customer_id (PK), balance – maintained in the same DB transaction as every charge, payment and reversal (fast lists; aging still calculated from detail)
customer_price	cp_no (CP0001), customer_id, bottle_type_code, price, standard_at_set (standard price when agreed, for reference), valid_from, valid_until (null = open), reason, customer_quotation_id (null unless set from a quotation), cancelled, cancel_reason – index (customer_id, bottle_type_code, valid_from). Rule: at most one non-cancelled row covers any date for a customer + bottle type (enforced by the service: a new row closes the previous one the day before)

Inventory & ledger

Table	Key columns
stock_balance	PK (bottle_type_code, bucket) – bucket: EMPTY / FACTORY / FILLED / CUSTOMERS / WRITTEN_OFF; qty CHECK (≥ 0) (BR-12 enforced in the DB too)
ledger_effect	source_type (SALES_TXN / GRN / BATCH / RETURN / DAMAGE / ADJUSTMENT / IMPORT), source_id, target (STOCK / CUSTOMER_BOTTLE / PRODUCT), bottle_type_code, product_code, customer_id, field (bucket or HELD / TO_COLLECT / OWED / STOCK), delta
stock_movement	movement_date, source_type, source_id, doc_no, item_code, bucket, qty_delta, description – history for the Movements tab and reports
damage_record	damage_date, bottle_type_code, qty, responsibility (COMPANY / CUSTOMER), location, reason, note, customer_id, source_type, source_id, reversed

Table	Key columns
stock_adjustment	adj_no, adj_date, item_code, bucket, mode (COUNT / LOST), system_qty, counted_qty, delta, reason
min_stock_level	bottle_type_code (PK), min_filled

Sales

Table	Key columns
sales_txn	bill_no (unique, B000001), paper_bill_no, kind (FIRST_PURCHASE / EXCHANGE / LEFT_AT_DOOR / COLLECT_EMPTIES / EXTRA_ORDER / PRODUCT_SALE / CUSTOMER_DAMAGE / OWED_SETTLEMENT / OPENING_BALANCE / REVERSAL), txn_date, customer_id, walk_in_name, amount, paid_on_bill, pay_method, water_amount, deposit_amount, product_amount, product_cost, note, phone_confirmed, damage_mode (REPLACE / REDUCE), settle_method (RETURN / DEPOSIT), credit_override_by, reversal_of_id, reversed_by_id
sales_txn_line	txn_id, bottle_type_code or product_code, filled, empties, unit_price, deposit_unit, deposit_count, old_bottles, old_brand_id, new_bottles, owed_added, owed_cleared, to_collect_added, to_collect_cleared, damaged, qty, line_amount

Partial unique index for BR-13: CREATE UNIQUE INDEX ux_paper_bill ON sales_txn (upper(paper_bill_no)) WHERE paper_bill_no IS NOT NULL AND kind <> 'REVERSAL' AND reversed_by_id IS NULL;

Billing

Table	Key columns
customer_payment	receipt_no (RC-00001), customer_id, pay_date, amount, method (CASH / BANK_TRANSFER / CHEQUE), reference, bank, cheque_date, invoice_id, note, balance_before
monthly_invoice	invoice_no (INV-2609-0001), customer_id, period (2026-09), invoice_date, due_date, opening_balance, charges, paid_on_bills, payments, amount, total_due – unique (customer_id, period)
invoice_txn	invoice_id, txn_id

Purchasing

Table	Key columns
quotation_request	qr_no (QR-2026-001), request_date, required_by, note, status (OPEN / QUOTES_RECEIVED / SELECTED / CLOSED)
qr_item / qr_supplier	requested items + qty / suppliers asked
quotation	quote_no, qr_id, supplier_id, supplier_ref, quote_date, valid_until, delivery_days, notes, attachment_path, selected, selection_reason, selected_by, selected_on – unique (qr_id, supplier_id)
quotation_line	quotation_id, item_code, unit_price (null = not quoted)
purchase_order	po_no (PO-2026-0001), qr_id, quotation_id, supplier_id, po_date, expected_date, status (DRAFT / APPROVED / SENT / PARTLY_RECEIVED / RECEIVED / CANCELLED), notes, approved_by, approved_on, cancel_reason
po_line	po_id, item_type, item_code, qty, unit_price, received_qty, damaged_qty
po_status_history	po_id, status, ts, user, note
goods_receipt / grn_line	grn_no, po_id, received_date, delivery_note, value / item_code, received, damaged, unit_price
supplier_payment	sp_no, supplier_id, po_id, pay_date, amount, method, reference, note

Production

Table	Key columns
factory_dispatch	dispatch_no (FD-0001), factory_id, dispatch_date, vehicle_no, note – one entry for all bottle types
factory_batch	batch_no (FB-0001), dispatch_id, factory_id, dispatch_date, bottle_type_code, qty_sent, charge_per_bottle (copied at dispatch), vehicle_no, note – one per bottle type in the dispatch
factory_return	return_no, batch_id, return_date, filled_qty, rejected_qty, reject_reason, cost (= filled × batch charge), note
factory_payment	fp_no, factory_id, period, pay_date, amount, method, reference, note

Expenses

Table	Key columns
expense_type	name (unique), description, active
expense	expense_no (EX-0001), paid_date, for_month (DATE, first day of the month), expense_type_id, description, paid_to, bill_no, amount, method (CASH / BANK_TRANSFER / CHEQUE), reference, deleted, delete_reason, deleted_by – soft delete so the audit trail stays complete

Customer quotations

Table	Key columns
customer_quotation	cq_no (QT-2026-001), customer_id (null for a new organisation), org_name, contact, designation, address, area_id, phone, email, customer_type_id (for prices), quote_date, valid_until, subject, delivery_text, terms_days, conditions, status (DRAFT / SENT / ACCEPTED / REJECTED), sent_via (EMAIL / BY_HAND / POST / WHATSAPP), sent_on, accepted_on, reject_reason, prices_applied_on, copy_of_id
cq_line	quotation_id, item_kind (WATER / DEPOSIT / PRODUCT / OTHER), bottle_type_code, product_code, description, qty, unit, unit_price, current_price (price the customer paid when quoted)
cq_history	quotation_id, status, ts, user, note

"Expired" is not stored – it is shown when status = SENT and valid_until < today.

Integrations & migration

Table	Key columns
whatsapp_message	customer_id, type (LEFT_AT_DOOR / PAYMENT_REMINDER / MONTHLY_INVOICE), template, to_number, params (JSON), status (QUEUED / SENT / DELIVERED / READ / FAILED / SKIPPED), meta_message_id, error, source_type, source_id, attempts, sent_at
import_batch	type (CUSTOMERS / OPENING_STOCK), file_name, rows_ok, rows_rejected, status, result (JSON)

5. Core engines

These shared services hold the business rules. Modules call them; they never re-implement the rules.

5.1 Ledger Engine (ledger)

Every change to stock, customer bottle balances or product stock is a **posting** made of **effects**.

```

post(source, effects[]):
  1. Lock affected rows in a fixed order (stock_balance by bottle+bucket, customer_bottle_balance, product)
    using SELECT ... FOR UPDATE → no race conditions between two users.
  2. For each effect: new = current + delta. If any new < 0 → throw STOCK_NEGATIVE (BR-12) with a
    clear message, e.g. "Filled in store (20L) cannot go below zero - available 5, needed 8.
    Record a stock adjustment first."
  3. Apply all deltas, insert ledger_effect rows and stock_movement rows.
  4. Everything happens inside the caller's single DB transaction (all-or-nothing).

reverse(source): build the negated effects of the original, run post() (same checks), link original ↔ reversal.

```

Effects per transaction type (F = filled given, E = empties back, O = old bottles handed in, Q = qty):

Transaction	Stock effects (per bottle type)	Customer bottle effects
First purchase	FILLED -Q, CUSTOMERS +Q, EMPTY +O	held +Q
Exchange	FILLED -F, EMPTY +E, CUSTOMERS +(F-E)	owed / to_collect per 5.2
Left at door	FILLED -F, CUSTOMERS +F	to_collect +F
Extra order	FILLED -F, EMPTY +(E+O), CUSTOMERS +(F-E)	held +new bottles
Collect empties	EMPTY +Q, CUSTOMERS -Q	to_collect -Q
Customer damage – replace	CUSTOMERS -Q +Q, WRITTEN_OFF +Q, FILLED -Q	–
Customer damage – reduce	CUSTOMERS -Q, WRITTEN_OFF +Q	held -Q
Owed settled – bottles returned	EMPTY +Q, CUSTOMERS -Q	owed -Q
Owed settled – deposit paid	–	owed -Q, held +Q
Product sale	product stock -Q	–
Goods receipt	EMPTY +(received – damaged) or product +(received – damaged)	–
Factory dispatch	EMPTY -Q, FACTORY +Q	–
Factory return	FACTORY -(filled + rejected), FILLED +filled, WRITTEN_OFF +rejected	–
Company damage	(EMPTY / FILLED / FACTORY) -Q, WRITTEN_OFF +Q	–
Adjustment – lost	bucket -Q, WRITTEN_OFF +Q	–
Adjustment – count	bucket ±(counted – system)	–

Invariant (tested): $\text{stock_balance}[\text{bt}, \text{CUSTOMERS}] = \sum \text{customers} (\text{held} + \text{to_collect} + \text{owed})$ for every bottle type.

5.2 Bottle exchange rules (sales)

For each bottle type on an **exchange** bill:

1. $\text{diff} = F - E$.
2. If $\text{diff} > 0 \rightarrow \text{owed} += \text{diff}$. A note is required (BR-07, FR-38).
3. If $\text{diff} < 0 \rightarrow \text{excess} = -\text{diff}$. First clear **to_collect** (empties left at the door last trip, FR-36), then **owed**. Any excess still left $\rightarrow$ error "customer returned more empties than they hold".
4. Filled bottles of a type the customer does not hold $\rightarrow$ error "use First purchase / Extra order".
5. Amount = $F \times \text{water price (5.5)}$. Cash paid is recorded on the bill.

Left at door: empties forced to 0; to_collect += F; phone agreement **and** note required (BR-04). **First purchase:** customer must hold 0 of that bottle type; $O \leq Q$; the brand must be accepted for that bottle size; amount = $Q \times \text{price} + (Q - O) \times \text{deposit}$ (BR-01, BR-02). **Extra order:** new bottles = $\max(0, F - E)$; $E \leq F$; $E \leq \text{held}$; $O \leq \text{new}$

bottles; deposits = new – O (FR-39). **Customer damage:** REPLACE → charge deposit × Q (+ water × Q if a filled replacement is given); REDUCE → no charge, holding reduced; damaged bottle written off and a `damage_record` created (FR-37, BR-06). **Owed settlement:** RETURN → bottles back to empty stock; DEPOSIT → charge deposit × Q, the customer keeps the bottles (holding grows). **Product sale:** walk-in sales must be paid in full (FR-40).

5.3 Numbering Service (`numbering`)

`next(name)` locks the `doc_sequence` row (`SELECT ... FOR UPDATE`), returns the formatted number and increments it – in the same DB transaction as the document (the service refuses to run without one), so numbers are unique and gap-free. `next(name, date)` uses the period of `date` (e.g. the invoice month); yearly / monthly sequences only move forward – a period earlier than the current one is refused (`SEQUENCE_PERIOD_CLOSED`) because its numbers may already be used.

Sequence	Format	Reset
Customer	<code>C0001</code>	never
Product	<code>P01</code>	never
Supplier / Factory	<code>S01 / F01</code>	never
Bottle type	<code>B</code> + litres (e.g. <code>B20</code>) – derived, duplicate size rejected	–
Quotation request	<code>QR-2026-001</code>	yearly
Supplier quotation	<code>SQ-0001</code>	never
Customer quotation	<code>QT-2026-001</code>	yearly
Purchase order	<code>PO-2026-0001</code>	yearly
Goods receipt	<code>GRN-0001</code>	never
Factory dispatch / batch / return / payment	<code>FD-0001 / FB-0001 / FR-0001 / FP-0001</code>	never
Bill (all sales txns incl. reversals)	<code>B000001</code>	never
Receipt	<code>RC-00001</code>	never
Monthly invoice	<code>INV-YYMM-0001</code>	monthly
Stock adjustment / Supplier payment	<code>ADJ-0001 / SP-0001</code>	never
Expense / Customer price	<code>EX-0001 / CP0001</code>	never

5.4 Schedule Service (`delivery`)

Rule	Definition
First delivery (BR-16)	The very next <code>delivery_day</code> after the registration date (editable by the user).
Is due on date d	Customer ACTIVE, <code>d ≥ registered_on</code> , <code>weekday(d) = delivery_day</code> , and <code>(d - next_delivery_date) mod (7 × cycle_weeks) = 0</code> .
Due list for d (FR-42)	All customers due on d; if d is today, also customers whose <code>next_delivery_date < today</code> (missed).
Entered	A non-reversed EXCHANGE / LEFT_AT_DOOR / FIRST_PURCHASE / EXTRA_ORDER txn for the customer on d.
Advance (FR-46)	After an exchange / left-at-door / first purchase on date t: <code>n = next_delivery_date</code> ; while <code>n ≤ t</code> : <code>n += 7 × cycle_weeks</code> . A back-dated bill earlier than <code>next_delivery_date</code> does not move it. Reversals do not move it back.
Next / week after (FR-58)	From date d: first aligned date > d, and that date + cycle.

Rule	Definition
Last delivery (FR-58)	Latest non-reversed delivery txn dated before d.
Bottles needed (FR-45)	Σ usual quantity of customers due and not yet entered, per day; "needed before d" = sum from today to d - 1.

5.5 Pricing Service (masterdata)

Every price on a bill comes from this one service – screens, previews, Enter Bills, registration, quotations and imports never look up prices themselves.

- `waterPrice(customer, bottleType, date)` (BR-08, FR-05):
 - the customer's **agreed price**: the non-cancelled `customer_price` row for (customer, bottle type) with `valid_from ≤ date` and (`valid_until` is null or `≥ date`); if several, the latest `valid_from`;
 - otherwise the **standard price**: latest `price_entry` (WATER, bottle type, customer's type) with `effective_from ≤ date`.
- `isAgreed(customer, bottleType, date)` – true when step 1 found a row (used for the "agreed" badge on every screen).
- `standardPrice(bottleType, customerType, date)` – step 2 only (shown beside agreed prices, quotation hints, Sales report comparison).
- `deposit(bottleType, date)` = latest DEPOSIT entry with `effective_from ≤ date` – the same for every customer (BR-18).
- **Setting an agreed price** (Admin only): `setCustomerPrice(customer, bottleType, price, from, until?, reason, quotationId?)` – `price > 0`, reason required, `until ≥ from`; rows of the same customer + bottle type that start on/after `from` (and before `until`) are cancelled; a row covering `from` is closed at `from - 1`. **Back to standard**: `endCustomerPrice(customer, bottleType, from, reason)` closes / cancels the same way. Both audited with old → new price and the standard price.
- Prices are stored on each `sales_txn_line` (`unit_price`, `deposit_unit`), so past bills keep their price whatever changes later (FR-07). Future-dated standard or agreed prices are "scheduled" and apply automatically from their date – no job needed.
- Guide values for the price dialog (not rules): bottles a month = usual qty × 52 / 12 / cycle; filling charge of the main factory; margin a month = (price – filling charge) × bottles a month.

5.6 Account, credit & aging (billing)

Item	Rule
Charge	Each sales txn adds <code>amount - paid_on_bill</code> to <code>customer_account.balance</code> ; payments subtract; reversals add the negated values.
Credit check (BR-09, FR-48)	If <code>pay_type ≠ CASH</code> and <code>credit_limit > 0</code> and <code>balance + amount - paid > credit_limit</code> → reject with <code>CREDIT_LIMIT</code> unless the request carries <code>approveOverLimit = true</code> and the user is ADMIN; the approver is stored in <code>credit_override_by</code> and audited.
Aging (FR-51)	Per customer: charges = txns with net > 0 (by date); credits = payments + txns with net < 0. Credits settle the oldest charges first. Remaining charges are bucketed by days since txn date: 0–30, 31–60, 61–90, 90+. Over-payment shows as a negative balance (advance).
Monthly invoice (FR-49)	For a Monthly bill customer and period: txns in the period (excluding opening balances), opening = balance before the period, payments in the period, <code>amount = Σ(amount - paid)</code> , <code>total_due</code> = closing balance. One invoice per customer per period. Due date = invoice date + terms.

5.7 Audit (audit)

`AuditService.log(action, entity, ref, details)` is called explicitly from services (not AOP), inside the same transaction, with readable details. Examples: "Exchange for W.M. Sunil Perera: 20L filled 2 / empties 2, Rs. 700.00"; "Status → Approved"; "Over-limit sale approved by Nimal Perera". Logins and logouts are logged too. Audit rows are never updated or deleted (FR-57).

5.8 Validation & errors

- Request validation (Bean Validation) → `400 VALIDATION` with field errors.
- Business rule failures → `422` with `{ "code": "STOCK_NEGATIVE | CREDIT_LIMIT | DUPLICATE_PAPER_BILL | NOTE_REQUIRED | INVALID_STATE | ...", "message": "...", "details": {...} }`. Messages reuse the prototype wording.
- Optimistic lock conflict → `409 CONFLICT` ("This record was changed by someone else – reload").

6. Module designs

Short design notes per module. Detailed tasks are in the Master Task Breakdown.

Module	Design notes
Administration	Users CRUD in pop-ups; roles fixed (ADMIN, ACCOUNTANT, DELIVERY_STAFF); "Reset password" sets a temporary password + <code>must_change_password</code> ; deactivation bumps <code>token_version</code> (logs the user out). Roles & access tab is read-only, from the permission matrix (8.2). Audit log tab with filters and paging.
Master data	Tabs: Bottle types · Other products · Prices & deposits · Accepted old bottles. Price change = new <code>price_entry</code> with effective date and reason; "Price history" pop-up. The Prices tab also lists customer agreed prices (in force + scheduled, or all) with the standard price and % difference, linking to the customer. Bottle code from size. Product stock not editable here (only via receipts, sales, adjustments).
Customers	List with server-side search and filters (name, phone, code, area, day, type, pay type, status) and paging. Register pop-up = customer details + delivery + payment + "Bottles given on the registration date"; one API call creates the customer and the first-purchase bill(s) in one transaction, then returns bills for printing. Side panel = balances, delivery info, water price ("agreed" badge + standard), tabs History · Payments · Invoices · Prices , actions incl. Prices (agreed-price dialog, Admin) and Quotation. Register pop-up has an optional agreed price per bottle type + reason (applied from the registration date, before the first-purchase bill). Edit pop-up has a Status section (Active / Inactive + from date + reason list) and "Change prices..."; list rows have Deactivate / Reactivate; list filters include status and price (standard / agreed). Deactivation warns about bottles, empties, owed bottles and balance still on the account; reactivation moves a past next-delivery date forward. Customer types managed in a pop-up.
Suppliers & Factories	Tabs: Suppliers (items picked from the item list, terms from the fixed list) · Filling factories (charge per bottle type, terms, active).
Inventory	Stock table (5 buckets per bottle type) with editable minimum level; product stock; "Record damage" (company or customer) and "Stock adjustment" pop-ups; Damage & adjustments tab; Movements tab (paged). Low-stock banner (FR-31).
Sales – Enter Bills	Keyboard grid. On load / date change: <code>GET /delivery/due?date=</code> fills rows with usual quantities (auto rows). Live preview per row via <code>POST /sales/bills/preview</code> (debounced) or a client-side mirror of 5.2 for speed; the server re-validates on save . "Save all" posts each row independently (<code>POST /sales/bills</code>) – good rows save, bad rows show their error; over-limit rows offer "Approve & save" to Admin. Paper bill number required and unique (BR-13).
Sales – Sales & Bills	Bill list (search by system or paper number, customer, type, dates; paged). Bill details pop-up: lines, effects, user, Print, Reverse (with reason). "+ New" menu: First purchase, Extra order, Product sale, Collect empties, Customer-damaged bottle, Settle owed – each a pop-up with live total and the customer card ("after this bill").

Module	Design notes
Delivery	Daily Delivery List (date, area; print sheet; upcoming bottle requirement tab). Delivery Planning (Admin): date (default tomorrow), area, due/all switch; KPIs; ready check per bottle type; customers grouped by area with last / this date / next / week after; load sheet PDF.
Purchasing	Tabs: Quotations · Purchase orders · Supplier payments. QR pop-up auto-ticks suppliers of the chosen items. QR detail = side-by-side comparison; "Choose" (reason, not if expired); "Create PO" (only from the chosen quotation). PO pop-up with step bar and only the next action: Approve → Mark as sent → Receive goods. GRN pop-up (received / damaged per line). Supplier owed = value of good items received – payments.
Production	Batches table (with dispatch number, Print dispatch note); "Send empties" = one dispatch for all bottle types (factory, date, vehicle, note + one quantity row per bottle type with empty-after, at-factory-after, charge and cost; all lines validated together and saved in one transaction; one batch per bottle type); "Receive filled" (partial returns allowed, rejects need a reason and become company damage, cost = filled × batch charge). Factory payments tab: cost by factory and month, record payment against factory + month.
Billing	Tabs: Customer accounts (aging per customer, filters, Receive, Statement) · Monthly invoices (choose month, generate for Monthly bill customers without an invoice, view/print). Recent payments and cash collected list. Receipt print after payment.
Reports	One page: choose report + date range → KPIs + tables (+ chart for sales and expenses). Export PDF / Excel. Accountant sees Sales, Purchasing, Outstanding & aging, Profit, Expenses. Sales adds "customers on agreed prices": water billed vs standard price on the same dates (from <code>sales_txn_line.unit_price</code> vs <code>standardPrice</code>). Profit adds running expenses by type (expenses whose <code>for_month</code> is between the From and To months) and net profit . New Expenses report (type × month, all entries).
Dashboard	KPIs (sales today, collected today, filled 20L, at factory, deliveries today entered/due, empties to collect, bottles owed, outstanding), low-stock banner, today's deliveries, "needs attention" list (also: customer quotations not sent / expiring within 7 days / accepted but not registered or prices not applied; agreed prices ending within 30 days; no expenses entered for last month), 14-day sales chart. One endpoint <code>GET /dashboard</code> .
Expenses	Month selector, type filter, search; KPIs (month total, previous month and difference, entries, biggest type); tabs Monthly expenses (list + by-type bars) · Month by month (type × last 6 months, Excel). Add / edit pop-up with "Save & add another"; delete with reason (soft delete + audit); expense types pop-up; print month statement. Admin and Accountant full.
Customer quotations	List with KPIs, search and status filter (Expired derived). New / edit / copy pop-up: to an existing customer (picker) or a new organisation; customer type for prices; line table (water / deposit / product / other) – unit price defaults to <code>waterPrice(customer)</code> or <code>standardPrice(type)</code> and follows the type until changed by hand; live total. View pop-up with step bar, history and only the next action: Mark as sent (via + date) → Accepted / Not accepted (reason). Print (PDF) and Email (<code>mailto:</code> with subject and summary; user attaches the PDF). Accepted → Register as customer (opens the register pop-up pre-filled, incl. quoted water prices as agreed prices; links <code>customer_id</code>) or Apply prices for an existing customer (<code>setCustomerPrice</code> per water line that differs, <code>prices_applied_on</code>). Admin full, Accountant view.
WhatsApp	See section 9. Settings tab in Administration (enable, phone number ID, business account ID, access token – stored encrypted, message toggles, test send). Message log list.
Data migration	Upload CSV → validate → preview (ready rows / rejected rows with reasons) → confirm → import in one transaction; <code>import_batch</code> record + audit. Menu hidden by the <code>feature.dataMigration</code> setting until go-live.

7. API design

7.1 Conventions

- Base path `/api` (Spring Boot context path; Nginx passes `/api/...` through unchanged). JSON. ISO dates (`2026-10-05`) in the API; the UI shows DD/MM/YYYY.

- Lists: `?page=0&size=50&sort=name,asc&q=...` → `{ items, page, size, total }`.
- Actions on documents are explicit sub-resources (`POST /purchase-orders/{id}/approve`) – no generic status PATCH.
- Every endpoint is protected by a permission (8.2) with `@PreAuthorize`.
- `POST .../preview` endpoints calculate totals and balances without saving (used by the pop-ups).

7.2 Endpoints

Module	Endpoints	
Auth	<code>POST /auth/login</code> · <code>POST /auth/logout</code> · <code>GET /auth/me</code> (user, role, permissions, menu) · <code>POST /auth/change-password</code>	
Admin	<code>GET/POST /users</code> · <code>PUT /users/{id}</code> · <code>POST /users/{id}/activate</code> · <code>/deactivate</code> · <code>/reset-password</code> · <code>GET /audit-log</code> · <code>GET/PUT /settings/{group}</code>	
Master data	<code>GET/POST /bottle-types</code> · <code>PUT /bottle-types/{code}</code> · <code>GET/POST /products</code> · <code>PUT /products/{id}</code> · <code>GET/POST /customer-types</code> · <code>PUT /customer-types/{id}</code> · <code>GET/POST /areas</code> · <code>GET /prices/current</code> · <code>GET /prices/history</code> · <code>POST /prices</code> · <code>GET/POST /old-bottle-brands</code> · <code>PUT /old-bottle-brands/{id}</code>	
Partners	<code>GET/POST /suppliers</code> · <code>PUT /suppliers/{id}</code> · <code>GET/POST /factories</code> · <code>PUT /factories/{id}</code>	
Customers	<code>GET /customers</code> · <code>GET /customers/{id}</code> (incl. balances) · <code>GET /customers/{id}/history</code> · <code>POST /customers/register</code> (customer + bottles given) · <code>POST /customers/register/preview</code> · <code>PUT /customers/{id}</code> · <code>POST /customers/{id}/deactivate</code> (date, reason) · <code>/reactivate</code> · <code>GET /customers/lookup?q=</code> (picker) · <code>GET /customers/{id}/prices</code> (history) · <code>POST /customers/{id}/prices</code> (set agreed price) · <code>POST /customers/{id}/prices/end</code> (back to standard) · <code>GET /customer-prices?status=current</code>	<code>all</code> (list for Prices tab) · <code>GET /pricing/water?</code> <code>customerId=&bottleType=&date=</code> (price + agreed flag + standard)
Sales	<code>POST /sales/bills</code> (exchange / left at door) · <code>POST /sales/bills/preview</code> · <code>POST /sales/first-purchase</code> · <code>/extra-order</code> · <code>/product-sale</code> · <code>/collect-empties</code> · <code>/customer-damage</code> · <code>/settle-owed</code> (each with <code>/preview</code>) · <code>GET /sales/txns</code> · <code>GET /sales/txns/{id}</code> · <code>POST /sales/txns/{id}/reverse</code> · <code>GET /sales/paper-bill/{no}</code> (duplicate check)	
Delivery	<code>GET /delivery/due?date=&area=</code> · <code>GET /delivery/planning?date=&area=&show=due</code> (or <code>show=all</code>) · <code>GET /delivery/requirement?days=14</code>	
Inventory	<code>GET /stock</code> · <code>GET /stock/movements</code> · <code>POST /stock/damage</code> · <code>POST /stock/adjustments</code> · <code>GET /stock/damages</code> · <code>PUT /stock/min-levels/{bt}</code> · <code>GET /stock/alerts</code>	

Module	Endpoints
Purchasing	GET/POST /quotation-requests · GET /quotation-requests/{id} (with quotations) · POST /quotation-requests/{id}/quotations · POST /quotations/{id}/choose · POST /quotations/{id}/create-po · GET /purchase-orders · GET /purchase-orders/{id} · PUT /purchase-orders/{id} (draft only) · POST /purchase-orders/{id}/approve · /send · /cancel · POST /purchase-orders/{id}/receipts · GET /supplier-balances · GET/POST /supplier-payments
Production	POST /factory-dispatches (lines[{bottleType, qty}]) · GET /factory-dispatches/{no} · GET /factory-batches · POST /factory-batches/{id}/returns · GET /factory-balances · GET/POST /factory-payments
Expenses	GET /expenses?month=&type=&q= · POST /expenses · PUT /expenses/{id} · DELETE /expenses/{id} (reason in body; soft delete) · GET /expenses/summary?from=&to= · GET/POST /expense-types · PUT /expense-types/{id}
Customer quotations	GET /customer-quotations?status=&q= · GET /customer-quotations/{id} · POST /customer-quotations (draft; copyOf optional) · PUT /customer-quotations/{id} (draft only) · POST /customer-quotations/{id}/send · /accept · /reject · /apply-prices · POST /customers/register accepts fromQuotationId
Billing	GET /accounts (aging rows, filters) · GET /accounts/summary · POST /payments · GET /payments · GET /invoices?period= · GET /invoices/preview?period= · POST /invoices/generate
Print (PDF)	GET /print/bill/{txnId} · /receipt/{paymentId} · /invoice/{id} · /statement/{customerId}?from=&to= · /purchase-order/{id} · /delivery-sheet?date=&area= · /load-sheet?date=&area= · /dispatch-note/{dispatchNo} · /customer-quotation/{id} · /expenses?month=
Reports	GET /dashboard · GET /reports/{name}?from=&to= · GET /reports/{name}/export?format=pdf&from=&to= (or format=xlsx)
WhatsApp	GET /whatsapp/messages · POST /whatsapp/test · GET/POST /whatsapp/webhook (Meta verification + status callbacks, public, signature-checked)
Migration	POST /migration/customers/validate · POST /migration/customers/import · POST /migration/stock/validate · POST /migration/stock/import · GET /migration/batches

8. Security & roles

8.1 Authentication

- POST /auth/login checks username + password (BCrypt) and active status → sets the JWT cookie (HttpOnly; Secure; SameSite=Strict, 12 hours – one working day).
- The JWT holds user id, role and token_version. Each request checks the user is still active and the token_version matches (deactivation or password reset logs the user out).

- 5 failed logins → 15-minute lock for that username. All logins and logouts are audited.
- `must_change_password` → the UI forces the change-password screen after login; the server also refuses every call except `/auth/me`, `/auth/change-password` and `/auth/logout` (403 `PASSWORD_CHANGE_REQUIRED`). Changing the password ends the user's other sessions.
- Unknown username and wrong password give the same message; "deactivated" is shown only after the correct password.
- HTTPS only (Nginx + Let's Encrypt); HSTS; CORS not needed (same domain).

8.2 Permission matrix

Permissions are strings (e.g. `customers.view`, `customers.edit`). Roles map to permissions in code. The frontend builds the menu from `/auth/me` and hides buttons the user may not use; the backend enforces everything.

Area	Admin	Accountant	Delivery Staff
Dashboard	Full	View	–
Bottles & Products	Full	–	–
Customers	Full	View	–
Suppliers & Factories	Full	View	–
Purchasing	Full	View	–
Filling Factory	Full	View	–
Stock	Full	–	–
Enter Bills / Sales & Bills	Full	–	–
Delivery Planning	Full	–	–
Daily Delivery List	Full	–	View + print
Billing & Payments	Full	Full	–
Expenses	Full	Full	–
Customer Quotations	Full	View + print	–
Customer agreed prices	Set / change	View	–
Reports	All 10	Sales, Purchasing, Outstanding & aging, Profit, Expenses	–
Administration (users, settings, audit)	Full	–	–
Data Migration	Full (when enabled)	–	–
Approve sale over credit limit	Yes	–	–

Landing page: Admin / Accountant → Dashboard; Delivery Staff → Daily Delivery List.

9. Background jobs & integrations

9.1 Scheduled jobs (Spring `@Scheduled`, single instance)

Job	When	What
WhatsApp sender	Every 1 minute	Sends QUEUED messages (max 3 attempts, then FAILED)

Job	When	What
Payment reminders	Monday 09:00	Queues a reminder for each Credit / Monthly bill customer (opted in) with a balance older than their terms; at most one per customer per week
Login lock clean-up	Hourly	Clears expired login locks
Database backup	23:00 nightly	Host cron script (not Spring): <code>pg_dump</code> → 7 days on the server + copy to S3 (30 days)

Scheduled price changes need no job (5.5).

9.2 WhatsApp Business (Phase 1 – Option A)

- **Outbox pattern:** the business transaction (bill saved as Left at door, invoice generated, reminder job) inserts a `whatsapp_message` row (QUEUED) **in the same DB transaction** – only if WhatsApp is enabled, that message type is switched on, and the customer has opted in with a WhatsApp number. The sender job then calls the Cloud API, so a Meta outage never blocks billing.
- **Templates** (approved in Meta Business Manager; English, Sinhala optional later):

Template	Parameters
<code>left_at_door_v1</code>	customer name, bottles left, date, next delivery date
<code>payment_reminder_v1</code>	customer name, balance, due date
<code>monthly_invoice_v1</code> (document header)	customer name, month, total due, due date + invoice PDF uploaded as media

- **Webhook** `/api/whatsapp/webhook`: verify token on GET; on POST, check the `X-Hub-Signature-256` signature and update message status (SENT / DELIVERED / READ / FAILED).
- **Secrets** (access token, app secret) are stored encrypted in `app_setting` (AES key from an environment variable), never returned to the UI.
- **Cost control:** monthly count shown in the message log; the toggle can switch any type off.

9.3 Email

Not used in Phase 1 (BRD section 11). The design leaves a `notification` interface so SMTP can be added later without changing callers.

10. Printing & exports

Document	Endpoint	Layout (from prototype)
Bill	<code>/print/bill/{id}</code>	Company header, bill no + paper no, customer, lines (water, deposits, old bottles, empties), total, paid, balance now, bottles held, next delivery, signatures
Receipt	<code>/print/receipt/{id}</code>	Receipt no, customer, amount, method + reference, balance before / now
Monthly invoice	<code>/print/invoice/{id}</code>	Invoice no, period, due date, delivery lines, previous balance, payments, this month, total due; "VAT not applied" note until confirmed
Statement	<code>/print/statement/{cid}</code>	Brought forward, charges / payments / running balance, aging line
Purchase order	<code>/print/purchase-order/{id}</code>	PO no, supplier, lines, total, prepared/approved by; DRAFT / CANCELLED watermark
Delivery sheet	<code>/print/delivery-sheet</code>	A4 landscape, grouped by area, blank columns for bill no / filled / empties / cash

Document	Endpoint	Layout (from prototype)
Load sheet	/print/load-sheet	Route totals table + customer list by route
Dispatch note	/print/dispatch-note/{no}	Dispatch no, date, vehicle, factory, one line per bottle type (batch no, empties sent), total, signatures (sent by, driver, received by factory)
Customer quotation	/print/customer-quotation/{id}	QUOTATION, no, date, valid until, To (contact, organisation, address), subject, lines (description, qty, unit, unit price, amount), total, delivery, payment terms, numbered conditions, signatures; DRAFT banner while draft
Expense statement	/print/expenses?month=	Month, entries by type (date, description, paid to, method, amount), total, totals by type, prepared / checked / approved

- PDFs are generated on the server (OpenPDF) with a shared header (company name, address, phone, reg. no, logo from settings), DD/MM/YYYY dates and LKR amounts.
- Report exports: PDF (same renderer) and Excel (Apache POI) with the same columns as the screen.

11. Non-functional design

NFR	Design
NFR-01 Browsers	Next.js targets current Chrome, Edge, Firefox; tested in all three before UAT
NFR-02 Capacity (10,000 customers, hundreds of bills/day)	Server-side paging/filtering; indexes (below); <code>customer_account</code> balance table avoids summing history for lists; aging calculated per page of customers, report export streams rows
NFR-03 Speed (≤ 3 s)	Indexes; no N+1 queries (fetch joins / projections); dashboard served by aggregate queries; target < 1 s for common lists
NFR-04 Keyboard entry	Bill grid: Enter $\rightarrow$ next field, Enter on the last field $\rightarrow$ new row, Ctrl+S saves all, E/L keys for type; customer picker accepts code, phone or name
NFR-05 Security	Section 8
NFR-06 Backup	Section 12.3
NFR-07 LKR / DD/MM/YYYY	<code>MoneyText</code> and <code>DateField</code> components used everywhere
NFR-08 Mobile-ready	All logic behind REST API (P1)
NFR-09 Simple screens	Pop-ups / side panel / step bar components (3.3)

Main indexes: `customer(lower(name))`, `customer(phone)`, `customer(area_id, delivery_day, status)`, `customer(next_delivery_date)`, `sales_txn(customer_id, txn_date)`, `sales_txn(txn_date)`, `sales_txn(kind, txn_date)`, `ux_paper_bill` (4.3), `customer_payment(customer_id, pay_date)`, `stock_movement(movement_date)`, `stock_movement(item_code, movement_date)`, `audit_log(ts)`, `whatsapp_message(status)`.

Logging & monitoring: Spring Boot logs (JSON, daily rotation, 14 days); `/actuator/health` (internal only) used by Docker health checks; CloudWatch alarm on CPU > 80 % and disk > 80 %.

12. Environments, deployment & backup

12.1 Environments

Environment	Where	Data
Local	Developer laptop, Docker Compose	Seed data (from the prototype's sample data)

Environment	Where	Data
UAT	NUVI shared UAT EC2	Seed data, then a copy of migrated data for go-live rehearsal
Production	Thuhina Water's own EC2 t4g.small, AWS Mumbai	Live data

Configuration through environment variables (`.env` , not in Git): DB credentials, JWT secret, encryption key, S3 bucket, WhatsApp app secret, domain.

12.2 CI/CD

Branching: `feature/*` → `dev` → `main` (tagged). `main` is production; there is no separate `prod` branch. `hotfix/*` branches start from `main` and merge back into both `main` and `dev`. Full rules are in `Master_Task_Breakdown.md` section 2.

1. Pull request (to `dev` or `main`) → GitHub Actions: backend build + unit/integration tests (Testcontainers), frontend lint + type-check + build.
2. Merge to `dev` → build Docker images (`api` , `web`), push to the registry, deploy to UAT automatically.
3. Release PR `dev` → `main`, then tag `vX.Y.Z` on `main` → manual approval → deploy to production (`docker compose pull && docker compose up -d` ; Flyway migrates on start). Rollback = redeploy the previous tag.

12.3 Backup & restore (NFR-06)

Item	Setting
Nightly dump	23:00 <code>pg_dump -Fc</code> , kept 7 days on the server
Off-server copy	Copied to S3, kept 30 days (lifecycle rule)
Server snapshot	Weekly EBS snapshot, kept 4 weeks (AWS Data Lifecycle Manager)
Restore test	Monthly: restore the latest dump to UAT and compare key totals (customers, stock buckets, outstanding)

13. Testing strategy & definition of done

13.1 Test levels

Level	Tool	Must cover
Unit	JUnit 5	Every rule in sections 5.1–5.6 (exchange maths, left at door, first purchase, extra order, damage, owed, numbering format, schedule dates, price lookup incl. agreed price in force / scheduled / ended / cancelled and back-to-standard , aging FIFO, credit check), multi-line dispatch all-or-nothing, expense for-month, quotation status steps
Integration	Spring Boot Test + Testcontainers	Each API endpoint: success, validation error, permission denied, business error; DB constraints (no negative stock, unique paper bill)
Concurrency	Integration	Two simultaneous postings on the same stock row → no negative stock, no lost update
Invariants	Integration	After every test scenario: CUSTOMERS bucket = Σ (held + to_collect + owed); customer_account balance = Σ charges – payments
Frontend	Type-check, lint; component tests for DateField, MoneyText, bill grid keyboard flow	

Level	Tool	Must cover
End-to-end smoke	Playwright	The client's prototype test path: register customer with bottles (and an agreed price) → Enter Bills (shortage, left at door, duplicate paper bill, agreed price used) → change an agreed price (old bills unchanged) → delivery planning → send empties of two bottle types → collect empties → reverse → payment → monthly invoices → customer quotation accepted → register → expenses → Profit shows net profit
UAT	Client	BRD v4.0 requirement checklist

Reference scenarios: the prototype's seeded data and the end-to-end checks done during the prototype review (e.g. "3 × 20L with 1 American Water bottle = Rs. 3,050") are reused as expected results.

13.2 Definition of done (each module)

- ☐ Flyway migration for the module's tables, with constraints and indexes
- ☐ Services implement the rules from this document; no business logic in controllers or the frontend
- ☐ API endpoints with permissions, validation and error codes; listed in Swagger
- ☐ Unit + integration tests pass in CI; invariants hold
- ☐ Screen matches the prototype (layout, wording, behaviour) for each role
- ☐ Audit entries written for every create / change / approve / reverse
- ☐ Prints/exports work (if the module has any)
- ☐ Acceptance checklist in the task breakdown ticked

14. Coding conventions

Topic	Convention
Git	<code>main</code> (deployable) + short-lived <code>feature/M<nn>-<short-name></code> branches; PR review before merge
Commits	<code>M06: add exchange posting with owed/to-collect rules</code>
Java	Package-by-module; constructor injection; DTOs (records) at the API boundary, entities never exposed; <code>BigDecimal</code> for money with <code>RoundingMode.HALF_UP</code> , 2 decimals
SQL	snake_case tables/columns; Flyway files <code>V<nnn>__<module>_<what>.sql</code>
TypeScript	Strict mode; feature folders; API types generated from OpenAPI (<code>openapi-typescript</code>)
UI text	Plain English as in the prototype; amounts <code>Rs. 1,350.00</code> ; dates <code>DD/MM/YYYY</code>
Errors	Business errors return <code>code</code> + human message; UI shows the message as a toast or inline
Seed data	<code>server/src/main/resources/db/seed/</code> (local/UAT only), based on the prototype's sample customers, prices and stock

15. Traceability (modules ↔ BRD)

Module (task breakdown)	Main requirements
M0 Foundation	FR-56 (login), FR-57 (audit), FR-60 (numbering), NFR-05, NFR-07, NFR-09
M1 Administration	FR-56, FR-57, section 5
M2 Master data	FR-01–07 (standard prices + Pricing Service), BR-01, BR-02, BR-08
M3 Suppliers & factories	FR-14, FR-22, BR-11
M4 Inventory core	FR-27–31, BR-05, BR-12, BR-15

Module (task breakdown)	Main requirements
M5 Customers	FR-03, FR-05 (agreed prices), FR-08–13 (incl. status), BR-09, BR-16, BR-18
M6 Sales core	FR-32, FR-33, FR-35–41, FR-59, BR-01–07, BR-13, BR-14
M7 Enter Bills	FR-34–36, FR-38, FR-41, FR-46–48, NFR-04
M8 Delivery schedule	FR-42–46, FR-58
M9 Purchasing	FR-15–21, BR-10
M10 Production	FR-23 (multi-bottle dispatch)–26, BR-11
M11 Billing & payments	FR-47–52, BR-09
M12 Dashboard & reports	Section 8 (reports incl. agreed-price comparison, net profit), FR-31
M13 WhatsApp	FR-61
M14 Data migration & go-live	FR-53 (incl. agreed prices)–55, NFR-01–03, NFR-06
M15 Expenses	FR-62, FR-63, BR-19
M16 Customer quotations	FR-64–66, BR-20

16. Risks & open points

#	Item	Impact	Action
1	Meta WhatsApp account, number and template approval	M13 cannot go live without them	Client starts Meta Business verification early; build M13 with a sandbox number
2	Actual prices, 10L deposit, filling charges, credit limits	Seed data only	Client to confirm before UAT (BRD section 10)
3	VAT on invoices	Invoice layout and totals	Client to confirm; invoice template has a VAT line ready but hidden
4	Bill / invoice / PO layout and logo	Print templates	Client to supply logo and company details; layouts follow the prototype until then
5	Quality of the client's Excel data for import	Go-live date	Run a trial import on UAT early (M14) and fix rejected rows with the client
6	t4g.small memory (2 GiB) for Next.js + JVM + PostgreSQL	Performance	Cap JVM heap at ~768 MB; resize to t4g.medium if needed (no code change)
7	Agreed prices need discipline (many customers on many prices)	Wrong prices on bills, lower margin	Admin-only, reason required, audited; Prices tab list and Sales report comparison for the owner's review; dashboard warns before agreements end
8	List of current agreed prices from the client	Bills wrong after go-live	Import agreed prices with the customers (FR-53) and check them in UAT
9	Deposit for very large customers	Possible later change	v4.0 keeps one deposit for all (BR-18); client to confirm

Next document: **Master Task Breakdown** (*Master_Task_Breakdown.md*) – modules M00–M16, each with server and client tasks and an acceptance checklist, built on this design.